MÓDULO: LENGUAJE DE MARCAS

CURSO: DAM | IFC32A

POKÉMON TEAM BUILDER

Documentación del proyecto

Alumno: Álex De Dios Pallicer

Profesor: Marc Reus Martí

Fecha: 12/04/2026

Índice

1. IDEA GENERAL DEL PROYECTO 2
2. ESTRUCTURA BÁSICA DEL TRABAJO 2
3. DATOS MOCKEADOS UTILIZADOS 2
4. PRINCIPALES DECISIONES DE DISEÑO Y ORGANIZACIÓN 2
5. EXPLICACIÓN GENERAL DEL CÓDIGO Y FUNCIONALIDADES 3
6. USO DE LA IA 3

1. IDEA GENERAL DEL PROYECTO

El proyecto es una aplicación web interactiva de una sola página que funciona como una Pokédex y un creador de equipos (*Team Builder*). Permite al usuario explorar un catálogo de 80 Pokémon clásicos (evoluciones finales o únicas de la primera generación), buscar por nombre, filtrar por tipo y seleccionar hasta 6 Pokémon para formar un equipo. Los cambios, como la actualización del equipo o los resultados de búsqueda, se reflejan en tiempo real en la interfaz.

2. ESTRUCTURA BÁSICA DEL TRABAJO

El proyecto se ha construido utilizando únicamente tecnologías web nativas (Vanilla), estructuradas en tres capas:

- **HTML5**: Define la estructura semántica de la página. Se divide en un `<header>` principal y un `.contenedor-app` que alberga dos paneles laterales: una `<section>` para el catálogo (Pokédex) y un `<aside>` para el equipo.
- **CSS3:** Se encarga del diseño visual y la disposición. Se ha prescindido de *frameworks* externos, utilizando variables CSS (`:root`) para la paleta de colores y el sistema de diseño.
- **JavaScript**: La lógica está separada en dos archivos para mayor limpieza: data.js (almacenamiento de datos estáticos) y app.js (manipulación del DOM y gestión de eventos).

3. DATOS MOCKEADOS UTILIZADOS

Los datos están estructurados en el archivo `data.js` mediante un array de objetos (`pokemonData`). Se han mockeado 80 Pokémon, cada uno conteniendo: un identificador numérico (`id`), el nombre (`name`), un array con sus tipos (`types`), sus estadísticas de combate (`stats`) y una URL (`image`). Para darle un aspecto moderno y dinámico, las URLs de las imágenes no apuntan a recursos locales estáticos, sino a GIFs animados de los modelos 3D alojados en el repositorio oficial de Showdown a través de la PokéAPI.

4. PRINCIPALES DECISIONES DE DISEÑO Y ORGANIZACIÓN

- **Layout Adaptativo**: Se ha utilizado CSS Flexbox para la estructura principal (.contenedor-app), permitiendo que el panel de la Pokédex (`flex: 2`) tenga más anchura que el panel lateral del equipo (`flex: 1`).

- **Grid Responsivo**: El catálogo de Pokémon utiliza CSS Grid con la regla `grid-template-columns: repeat(auto-fill, minmax(150px, 1fr))`, lo que garantiza que las tarjetas se adapten automáticamente a cualquier tamaño de pantalla sin necesidad de Media Queries complejas.
- **Navegación Fluida**: El panel lateral del equipo utiliza position: sticky; top: 20px;, lo que permite que el equipo seleccionado acompañe visualmente al usuario mientras hace scroll por el extenso catálogo.
- **Experiencia de Usuario (UX)**: Se han incluido enlaces dinámicos en las imágenes y nombres de los Pokémon que abren su página correspondiente en la WikiDex en una pestaña nueva (`target="_blank"`), evitando que el usuario pierda su progreso al consultar información. Además, los tipos de Pokémon están representados con badges coloreados (ej. rojo para fuego, verde para planta) para una identificación visual rápida.

5. EXPLICACIÓN GENERAL DEL CÓDIGO Y FUNCIONALIDADES

El archivo app.js gestiona el estado y la interactividad mediante las siguientes técnicas:

- **Renderizado Dinámico:** La función `renderListaPokemon` vacía el contenedor HTML y lo reconstruye iterando sobre el *array* de datos utilizando *Template Literals* (`) para inyectar el código HTML y las variables.
- **Gestión de Estado Centralizada:** Para evitar conflictos entre el buscador y los botones, se utilizan dos variables globales (`textoActualSeleccionado` y `filtroTipoSeleccionado`). La función `aplicarFiltro()` lee ambas variables y encadena el método `.filter()` sobre el *array* original para mostrar siempre el resultado correcto de la combinación de ambos filtros.
- **Delegación de Eventos:** Para mejorar el rendimiento, en lugar de asignar un *Event Listener* a cada botón individualmente, se asignan a los contenedores padre (`#lista-pokemon`, `#filtros`, `#slots-equipo`). Mediante event.target.classList.contains() se intercepta el clic exacto, extrayendo los datos necesarios mediante atributos personalizados (`data-id`, `data-type`, `data-index`).

6. USO DE LA IA

La inteligencia artificial se ha utilizado como asistente de programación durante el desarrollo del proyecto.

- **¿Para qué se ha utilizado?** Para generar el extenso *array* de datos de prueba (data.js), estructurar la lógica inicial de JavaScript, y como herramienta de depuración para identificar y corregir errores de sintaxis o de nombres de variables durante la traducción del código al español.
- **¿En qué parte ha influido?** Principalmente en la implementación de técnicas avanzadas como la delegación de eventos en JS, el uso de CSS Grid/Flexbox y la resolución de conflictos de estado entre los filtros de búsqueda y los botones de tipo.
- **¿Qué se ha adaptado?** El código sugerido se ha adaptado al idioma español para mejorar su legibilidad y comprensión personal (miEquipo en lugar de myTeam, renderListaPokemon en lugar de renderGrid). Además, se ajustaron las propiedades CSS para asegurar que los elementos visuales (como la clase .active de los botones) se enlazaran correctamente con las interacciones del DOM.